

# Manual Operasional — QR Code Security System (RSA-PSS)

Panduan menjalankan sistem, mengeceknya, serta memantau & menghentikan proses. Server: Ubuntu, aplikasi berjalan via **systemd** (`qrcode.service`).

## 1. Arsitektur Singkat

```
Internet —> Nginx (443/80) —> Gunicorn (127.0.0.1:5000) —> Aplikasi Flask (wsgi:app)
                                     |
                                     — Redis (rate limit / state)
```

| Komponen      | Service systemd                   | Peran                                                                        |
|---------------|-----------------------------------|------------------------------------------------------------------------------|
| Aplikasi      | <code>qrcode.service</code>       | Gunicorn menjalankan <code>wsgi:app</code> (1 worker, 12 threads, port 5000) |
| Reverse proxy | <code>nginx.service</code>        | Meneruskan <code>https://rsa-pss.com</code> → <code>127.0.0.1:5000</code>    |
| Cache/state   | <code>redis-server.service</code> | Dependensi aplikasi                                                          |

**PENTING:** Aplikasi **HARUS** dijalankan lewat systemd. **JANGAN** menjalankan `python app.py` manual dari terminal — bila terminalnya ditutup, stdout menjadi tidak valid dan proses seperti kalibrasi gagal dengan `[Errno 5] Input/output error`.

## 2. Menjalankan & Mengelola Sistem

Semua perintah butuh `sudo`.

```
# Menyalakan aplikasi
sudo systemctl start qrcode

# Mematikan aplikasi
sudo systemctl stop qrcode

# Restart (mis. setelah update kode/template)
sudo systemctl restart qrcode

# Lihat status ringkas
sudo systemctl status qrcode

# Aktifkan auto-start saat server booting (sudah aktif)
sudo systemctl enable qrcode
```

```
# Matikan auto-start saat booting
sudo systemctl disable qrcode
```

Setelah **reboot server**, aplikasi + redis + nginx menyala otomatis (semua sudah **enabled**). Jika aplikasi crash saat berjalan, systemd otomatis menghidupkannya lagi (**Restart=always**, jeda 5 detik).

### 3. Melihat Log

```
# Ikuti log aplikasi secara real-time (Ctrl+C untuk keluar)
sudo journalctl -u qrcode -f

# Log 30 menit terakhir
sudo journalctl -u qrcode --since "-30 min" --no-pager

# Cari error tertentu
sudo journalctl -u qrcode --no-pager | grep -i "error\|traceback"
```

Log tambahan di dalam folder aplikasi: **logs/app.log**, **logs/server.log**.

### 4. Cek Sistem Berjalan & Sehat

```
# 1) Service aktif?
sudo systemctl is-active qrcode          # harus: active

# 2) Proses gunicorn ada?
pgrep -fa "gunicorn.*wsgi:app"

# 3) Port 5000 mendengarkan?
sudo ss -tlnp | grep :5000

# 4) Aplikasi merespons? (harus 200)
curl -s -o /dev/null -w "HTTP %{http_code}\n" http://127.0.0.1:5000/testing/calibration

# 5) Situs publik hidup? (harus 200/302)
curl -s -o /dev/null -w "HTTP %{http_code}\n" https://rsa-pss.com/
```

**Cek stdout sehat** (mencegah bug **[Errno 5]**) — harus menunjuk **socket:[...]**, **bukan pts (deleted)**:

```
sudo ls -l /proc/$(pgrep -f "gunicorn.*wsgi:app" | head -1)/fd/1
```

## 5. Cek Proses Background (Kalibrasi / Generate Massal / Verifikasi Massal)

Proses berat berjalan sebagai **thread di server**, bukan di browser. Menutup browser **tidak** menghentikannya — pekerjaan lanjut sampai selesai.

### 5a. Lewat Browser (cara utama)

Buka kembali halaman terkait; progress bar akan otomatis memuat status terkini dari server (via `task_id`). Halaman & endpoint pemantauan:

| Proses            | Halaman                           | Endpoint progress                               |
|-------------------|-----------------------------------|-------------------------------------------------|
| Kalibrasi         | <code>/testing/calibration</code> | <code>/testing/calibration_progress</code>      |
| Generate massal   | halaman generate                  | <code>/api/generate_progress_status</code>      |
| Verifikasi massal | halaman verifikasi massal         | <code>/api/verify_massal_progress_status</code> |

### 5b. Lewat Server (cek "apakah ada batch yang sedang jalan")

Batch yang aktif membuat worker unicorn sibuk (CPU tinggi). Ukur **CPU sesaat**:

```
CLK=$(getconf CLK_TCK)
read_ticks(){ local t=0; for p in $(pgrep -f "unicorn.*wsgi:app"); do read -ra S < /proc/$p/stat; t=$((t+${S[13]}+${S[14]})); done; echo $t; }
a=$(read_ticks); sleep 3; b=$(read_ticks)
echo "CPU sesaat worker: $(( (b-a)*100/(CLK*3) ))%"
```

- **~0%** → tidak ada batch berat yang jalan (idle).
- **80-100%** → ada proses (kalibrasi/generate/verify) sedang berjalan.

Catatan: `ps aux` menampilkan **rata-rata CPU seumur proses**, bukan sesaat — kurang akurat untuk menilai idle. Gunakan cara di atas.

## 6. Menghentikan Proses Background

1. **Cara aman (disarankan):** tekan tombol **Stop** di halaman prosesnya. Endpoint yang dipakai UI: `stop_calibration`, `/api/stop_generate_process`, `/api/stop_verify_massal_process`.
2. **Cara paksa:** `sudo systemctl restart qrcode` ⚠ Ini mematikan **SEMUA** proses background yang sedang jalan (kalibrasi/batch) di tengah jalan dan **tidak** dilanjutkan otomatis. Pakai hanya saat darurat atau saat yakin tidak ada batch penting berjalan (cek dulu dengan langkah 5b).

## 7. Stress Testing (Simulated vs Real HTTP)

Sistem punya **dua** mode uji beban yang menjawab pertanyaan **berbeda**. Keduanya bukan pembandingan langsung.

|                  | Simulated Stress Testing | Real HTTP Stress Test                       |
|------------------|--------------------------|---------------------------------------------|
| Cara kerja       | Model analitis di memori | Request HTTP sungguhan ke aplikasi          |
| Jaringan         | Tidak ada                | Ada (koneksi nyata)                         |
| Rate limit       | Tidak kena               | Kena (kecuali beban lokal, lihat bawah)     |
| Artefak          | Tidak membuat file       | Membuat QR, menulis log generate/verifikasi |
| Level user wajar | 100 – 1.500              | 10 – 100 (batas keras 500)                  |

### 7a. Kenapa level user Real HTTP lebih kecil

Karena dibatasi kapasitas nyata server, bukan sekadar angka di form:

- **CPU 2 core**, Unicorn **1 worker × 12 threads**.
- Generator beban berjalan **di dalam proses aplikasi yang sama**, sehingga penembak dan yang ditembak berebut CPU yang sama.
- Di atas ±100 user, yang terukur adalah **antrean dan timeout**, bukan performa kriptografi RSA-PSS.

**Workers wajib tetap 1.** `background_tasks` ( `app.py` ) dan `_calibration_state` ( `routes/testing_routes.py` ) disimpan di memori proses. Menambah worker membuat polling progres mendarat di worker yang tidak mengenal task tersebut → progres task/kalibrasi rusak. Untuk menambah kapasitas, naikan **threads** ( `GUNICORN_THREADS` ), bukan workers.

### 7b. Aturan penting: Requests per User Level

Konkurensi nyata per level = **min(concurrent users, operations)**.

Artinya **Requests per User Level harus ≥ level user tertinggi**, kalau tidak level tinggi tidak pernah benar-benar tercapai.

| Level user | Requests per level | Konkurensi nyata                       |
|------------|--------------------|----------------------------------------|
| 100        | 20                 | <b>20</b> ✗ (level 100 tidak tercapai) |
| 100        | 200                | <b>100</b> ✓                           |

Nilai default sekarang: level `10,25,50,100` dengan `200` requests per level.

### 7c. Base URL: lokal vs publik

| Base URL                                         | Efek                                                                                                                    |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>http://127.0.0.1:5000/</code><br>(default) | Langsung ke Unicorn. <b>Rate limit dikecualikan</b> , pengguna publik tidak terganggu. Dipakai untuk level user tinggi. |
| <code>https://rsa-pss.com/</code>                | Menyertakan overhead Nginx/HTTPS, tetapi <b>rate limit 60/menit aktif</b> → banyak HTTP 429.                            |

Pengecualian rate limit hanya berlaku bila request memenuhi **dua** syarat sekaligus: ber-`User-Agent: QRRealHTTPStress/1.0` **dan** berasal dari `127.0.0.1`. Klien dari luar tidak bisa memalsukannya, karena Nginx selalu menimpa `X-Forwarded-For` dengan IP asli pemanggil.

### 7d. Catatan untuk penelitian

Sajikan kedua mode sebagai pengukuran yang berbeda, bukan perbandingan angka langsung:

- **Simulated** → model skala besar (100–1.500 user).
- **Real HTTP** → validasi empiris end-to-end pada kapasitas server nyata.

Bila butuh konkurensi tinggi yang sah secara metodologis, beban harus ditembakkan dari **mesin terpisah** (mis. k6/locust dari host lain), bukan dari server yang sama.

## 8. Peringatan Penting

- ✗ **Jangan** `python app.py` manual → memicu `[Errno 5] Input/output error` pada kalibrasi.
- ⚠ **Restart service saat batch jalan** → batch berhenti di tengah, tidak resume. Selalu cek CPU (bagian 5b) sebelum restart.
- ⚠ **Setelah mengubah kode/template**, wajib `sudo systemctl restart qrcode` (template di-cache di memori karena `debug=False`).
- ● Service berjalan sebagai **root** (mengikuti kepemilikan file `data/`, `logs/`, DB).

## 9. Troubleshooting Cepat

| Gejala                       | Kemungkinan sebab                   | Tindakan                                                                            |
|------------------------------|-------------------------------------|-------------------------------------------------------------------------------------|
| Situs tidak bisa diakses     | qrcode/nginx mati                   | <code>sudo systemctl status qrcode nginx</code> lalu <code>start</code>             |
| HTTP 502 di browser          | gunicorn mati / port 5000 kosong    | <code>sudo systemctl restart qrcode</code> ; cek <code>ss -tlnp   grep :5000</code> |
| Kalibrasi gagal<br>[Errno 5] | app dijalankan manual (stdout mati) | Pastikan lewat systemd; cek fd (bagian 4)                                           |
| Perubahan kode tak muncul    | template/kode ter-cache             | <code>sudo systemctl restart qrcode</code>                                          |
| Ingin tahu sebab error       | —                                   | <code>sudo journalctl -u qrcode --since "-15 min"</code>                            |

## 10. Ringkasan Perintah (Cheat Sheet)

```

sudo systemctl start    qrcode    # nyalakan
sudo systemctl stop     qrcode    # matikan
sudo systemctl restart  qrcode    # restart
sudo systemctl status   qrcode    # status
sudo journalctl -u qrcode -f      # log real-time
pgrep -fa "gunicorn.*wsgi:app"    # cek proses
sudo ss -tlnp | grep :5000        # cek port
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:5000/testing/calibration

```